

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ОДЕСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ І.І.МЕЧНИКОВА
Факультет математики, фізики та інформаційних технологій
Кафедра математичного забезпечення комп'ютерних систем

КУРСОВА РОБОТА
з освітнього компоненту «Програмування»
на тему: СТВОРЕННЯ ІЄРАРХІЇ КЛАСІВ НА ТЕМУ «ЗАЛІЗНИЧНИЙ
ВОКЗАЛ»

Студента 1 курсу, денна форма навчання
спеціальність F7 «Комп'ютерна інженерія»

Прокопчука Івана Сергійовича

Керівник: к.т.н., доц. Шестопапов С.В.

Національна шкала: _____

Кількість балів: _____

ECTS: _____

Дата захисту: _____

Члени комісії _____ Антоненко О.С.

(підпис) (прізвище та ініціали)

_____ Шестопапов С.В.

(підпис) (прізвище та ініціали)

_____ _____
(підпис) (прізвище та ініціали)

ЗМІСТ

	стор.
ЗМІСТ.....	2
ВСТУП.....	3
ЗАВДАННЯ.....	4
1. ТЕОРЕТИЧНА ЧАСТИНА.....	5
1.1 Поняття ООП.....	5
1.2 Опис об'єктної частини.....	6
2. ПРАКТИЧНА ЧАСТИНА.....	7
2.1 Опис класів.....	7
2.2 Інтерфейс.....	10
ВИСНОВКИ.....	15
СПИСОК ЛІТЕРАТУРИ.....	16

ВСТУП

Мета роботи – створити ієрархію класів на тему «Залізничний вокзал» та застосувати її для виконання задачі, поданої в опису варіанту. Під час виконання роботи необхідно використати основні принципи об'єктно-орієнтованого програмування: поліморфізм, успадкування та інкапсуляцію.

Необхідно створити конструктори класів, поля та методи для роботи з об'єктами. Поля класів повинні бути закритими, а доступ до них має здійснюватися через відповідні властивості класу.

У програмі потрібно реалізувати ієрархію класів на основі відношень успадкування та композиції. Для реалізації динамічного поліморфізму слід використовувати віртуальні методи, абстрактні класи та віртуальні методи.

ЗАВДАННЯ

Варіант 5а. Створення ієрархії класів на тему «Залізничний вокзал»

Необхідно реалізувати систему класів, що описують структуру залізничного вокзалу, перони та поїзди. Передбачати можливість зберігання інформації про рейси, вибору конкретного потягу, вибору типу та номеру вагону та покупки квитка на цей рейс. Також треба реалізувати метод прибуття та відбуття потягу для можливості додавати ці потяги на вокзал.

Також необхідно реалізувати взаємодію між класами, перевірку коректності даних та виконання основних операцій, пов'язаних із функціонуванням вокзалу.

1. ТЕОРЕТИЧНА ЧАСТИНА

1.1 Поняття ООП

Об'єктно-орієнтоване програмування – це методологія програмування, заснована на представленні програми у вигляді сукупності об'єктів, кожний з яких є екземпляром певного класу, а класи утворюють ієрархію успадкування [1].

В ООП використовується 4 основні принципи:

- a) абстракція;
- b) інкапсуляція;
- c) успадкування;
- d) поліморфізм;

Абстракція – принцип, в якому об'єкти є спрощеним, ідеалізованим описом реальних сутностей предметної області [1].

Інкапсуляція (дослівно «укладання в оболонку») – це механізм, який об'єднує дані й методи, що маніпулюють цими даними, захищаючи те й інше від зовнішнього втручання або неправильного використання [1].

Успадкування – це механізм об'єктно-орієнтованого програмування, який дозволяє створювати новий клас на основі існуючого. Новий клас (похідний) отримує поля та методи базового класу, а також може додавати свої або перевизначати існуючі [2].

Поліморфізм – це здатність об'єктів з однаковим інтерфейсом вести себе по-різному залежно від конкретного типу. Він дозволяє використовувати один і той самий виклик методу для різних типів об'єктів [2].

1.2 Опис об'єктної частини

Програма пропонує різні функції для двох типів користувача: пасажир та диспетчера. Можливість виконання дій залежить від поточної ролі користувача.

Звичайний покупець (пасажир) взаємодіють з інтерфейсом Одеського вокзалу для перегляду табло з поточними потягами, що прибули на вокзал за допомогою методу `ArriveTrain()`. Пасажир може обрати рейс, тип та номер вагону на який він хоче купити квиток. Після чого пасажиру пропонується ввести свої дані, за допомогою яких створюється об'єкт `Ticket`(квиток, що пасажир викупив), та виводиться повна інформація квитка щодо потягу та персональних даних пасажирів.

Диспетчер вокзалу взаємодіє з інтерфейсом для можливості змінення табло вокзалу, за допомогою додавання потягів(метод `ArriveTrain()`) та видалення потягів(метод `DepartureTrain()`).

Незалежно від ролі користувача, інтерфейс виводить табло з поточними потягами, що знаходяться на перонах Одеського вокзалу.

2. ПРАКТИЧНА ЧАСТИНА

2.1 Опис класів

Діаграму класів наведено в додатку А. Нижче описанні класи проекту.

1) Клас «Station»:

- a) Приватні поля: name, platformAmount, platforms. trains.
- b) Конструктори:
 - Конструктор без параметрів
 - Конструктор з параметрами
- c) Властивості для полів: Name, PlatformAmount, Trains.
- d) Методи:
 - ArriveTrain – додає потяг на вокзал, розподіляючи його на вільний перон та займає цей перон.
 - DepartureTrain – видаляє потяг з вокзалу, звільнюючи перон, який він займав.
 - TrainsInfo – перебирає кожен потяг, повертаючи інформацію про них.
 - Перевизначення ToString – повертає інформацію про об'єкт(табло).

2) Клас «Train»:

- a) Приватні поля: startStation, finalStation, departureTime, arrivalTime, carAmount, platform, carriages.
- b) Конструктори:
 - Конструктор без параметрів
 - Конструктор з параметрами
- c) Властивості для полів: StartStation, FinalStation, DepartureTime, ArrivalTime, CarAmount, Platform, Carriages.
- d) Методи:

- AddCarriages – додає вагони заданої кількості кожного типу до складу потяга. compartmentCount – повертає кількість вагонів типу «Купе» у складі потяга.
- sittingCount – повертає кількість вагонів типу «Плацкарт» у складі потяга.
- CarriagesCount – повертає рядок з інформацією про кількість вагонів кожного типу.
- Перевизначення ToString – повертає інформацію про об'єкт (маршрут, час, перон).

3) Абстрактний клас «Carriage»:

- a) Захищені поля: totalPlaces, freePlaces, price.
- b) Конструктори: відсутні (визначаються у підкласах).
- c) Методи:
 - TotalPlaces – повертає загальну кількість місць у вагоні.
 - FreePlaces – повертає кількість вільних місць у вагоні.
 - ReducePlace – зменшує кількість вільних місць на одиницю при покупці квитка.
 - Абстрактний метод CarInfo – повертає інформацію про вагон.

4) Клас «CompartmentCar» (успадковується від Carriage):

- a) Приватні поля: успадковані від Carriage (totalPlaces, freePlaces, price).
- b) Конструктори:
 - Конструктор без параметрів
- c) Методи:
 - Перевизначення CarInfo – повертає інформацію про купе (місця, ціна).

5) Клас «SittingCar» (успадковується від Carriage):

- a) Приватні поля: успадковані від Carriage (totalPlaces, freePlaces, price).

b) Конструктори:

- Конструктор без параметрів

c) Методи:

- Перевизначення CarInfo – повертає інформацію про плацкарт (місця, ціна).

6) Клас «Ticket»:

a) Приватні поля: selectedTrain, selectedCar, carNum.

b) Конструктори:

- Конструктор з параметрами

c) Властивості для полів: SelectedTrain, SelectedCar, CarNum.

d) Методи:

- Перевизначення ToString – повертає інформацію про квиток (рейс, тип вагону, номер вагону).

7) Клас «Passanger»: Приватні поля: fullName, gender, age, id.

a) Конструктори:

- Конструктор з параметрами

b) Властивості для полів: FullName, Gender, Age, Id.

c) Методи:

- Перевизначення ToString – повертає інформацію про пасажир (ПІБ, стать, вік, номер паспорту).

2.2 Інтерфейс

Готовий проект розміщено в [4]. Під час запуску програми користувач бачить вступне вікно (Рис 2.1).

```
Виберіть дію:  
1. Одеський вокзал (купити квиток)  
2. Панель диспетчера
```

Рисунок 2.1 – початкове вікно

Користувач потрапляє сюди коли запускає програму. Після того як користувач введе номер дії, що хоче виконати, він побачить інший інтерфейс залежно від вибору.

```
Табло  
-----  
1. Одеса → Київ | Відправлення: 11:30 | Прибуття: 01:00 | Перон: 1  
2. Одеса → Львів | Відправлення: 21:15 | Прибуття: 07:30 | Перон: 2  
3. Одеса → Вінниця | Відправлення: 6:30 | Прибуття: 12:15 | Перон: 3  
-----  
  
Введіть номер потягу:  
|
```

Рисунок 2.2 – вікно покупки квитка

Після вибору 1. Одеський вокзал користувач бачить табло, на якому вказано інформацію щодо поточних потягів, та запропоновано ввести номер потягу, на який користувач хоче купити квиток (Рис 2.2).

```

Одеса → Київ | Відправлення: 11:30 | Прибуття: 01:00 | Перон: 1
-----
Кількість вагонів різних типів:
Купе: | Плацкорт:
7 | 8
-----

Введіть тип вагону:
1. Купе
2. Плацкорт
|

```

Рисунок 2.3 – інформація про вагони в вибраному потязі

Коли користувач обирає потяг, йому виводиться інформація про цей потяг (напрямок, час відправлення, час прибуття, перон), та кількість вагонів двох типів : купе та плацкорт. Також користувачу пропонується обрати, на вагоні якого типу він хоче подорожувати (Рис 2.3).

```

Введіть номер вагону
Всього: 7
|

```

Рисунок 2.4 – введення номеру вагону

Після вибору типу вагону, користувача просять ввести номер вагону та виводиться кількість вагонів цього типу (Рис. 2.4).

```

Введіть номер вагону
Всього: 7
1
Всього місць: 40
Вільних місць: 40
Ціна квитка: 800₴
Купити квиток?
1. Y 2. N
|

```

Рисунок 2.5 – інформація про вагон та підтвердження придбання

Коли користувач вводить номер вагону, програма виводить інформацію про цей конкретний вагон(кількість місць взагалі, кількість вільних місць, ціна квитка). Користувача просять підтвердити придбання квитка (Рис. 2.5).

Якщо користувач підтверджує придбання, його просять ввести свої персональні дані: ПІБ, стать, вік, та номер паспорту (Рис 2.6).

```
Введіть ПІБ:  
Прокопчук Іван Сергійович  
Введіть стать (М/F):  
m  
Введіть свій вік:  
19  
Введіть номер паспорту:  
123456789|
```

Рисунок 2.6 – введення персональної інформації пасажира

Якщо користувач вводить всі дані вірно, виводиться вікно з повною інформацією про придбаний квиток (Рис 2.7). Програма пропонує придбати ще один квиток.

```
Ваш білет:  
-----  
ПІБ: Прокопчук ?ван Серг?йович  
Стать: чоловік  
Вік: 19  
Номер паспорту: 123456789  
-----  
Рейс:  
Одеса → Київ | Відправлення: 11:30 | Прибуття: 01:00 | Перон: 1  
Тип вагону | Номер вагону  
Купе | 1
```

Рисунок 2.7 – квиток

Якщо ж користувач не підтвердить придбання квитку (введе N), то він отримає повідомлення «Покупку скасовано.», та програма запропонує купити інший квиток.

Для вибраного типу вагону «Плацкарт» функціонал програми повністю повторюється.

Якщо користувач обере на початковому екрані (Рис. 2.1) 2ий варіант Панель диспетчера, то побачить табло вокзалу та інтерфейс на якому він зможе обрати дію: додати потяг та видалити потяг (Рис. 2.8).

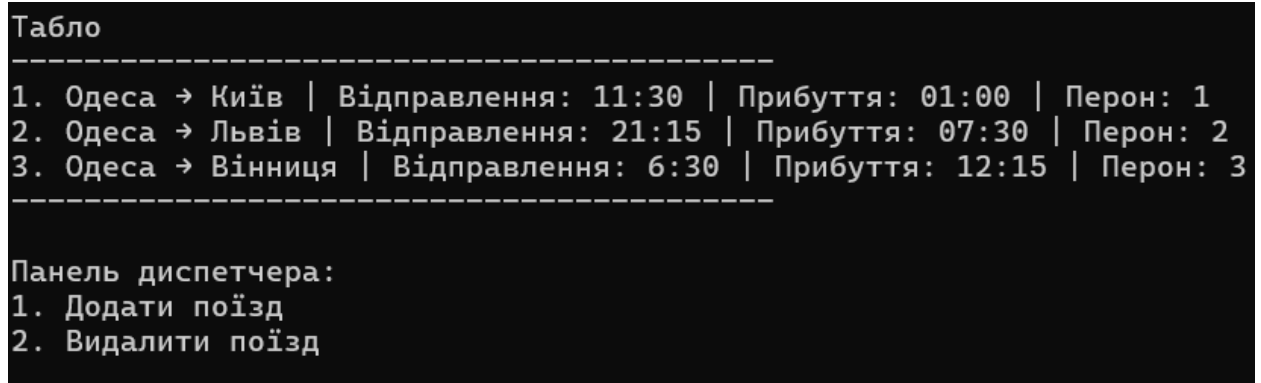


Рисунок 2.8 – панель диспетчера

Якщо користувач обере 1. Додати потяг, то йому буде запропоновано ввести інформацію про цей потяг, і, у випадку коли користувач вводить всі дані правильно, він отримає оновлене табло вокзалу. Програма пропонує купити ще один квиток (Рис. 2.9).

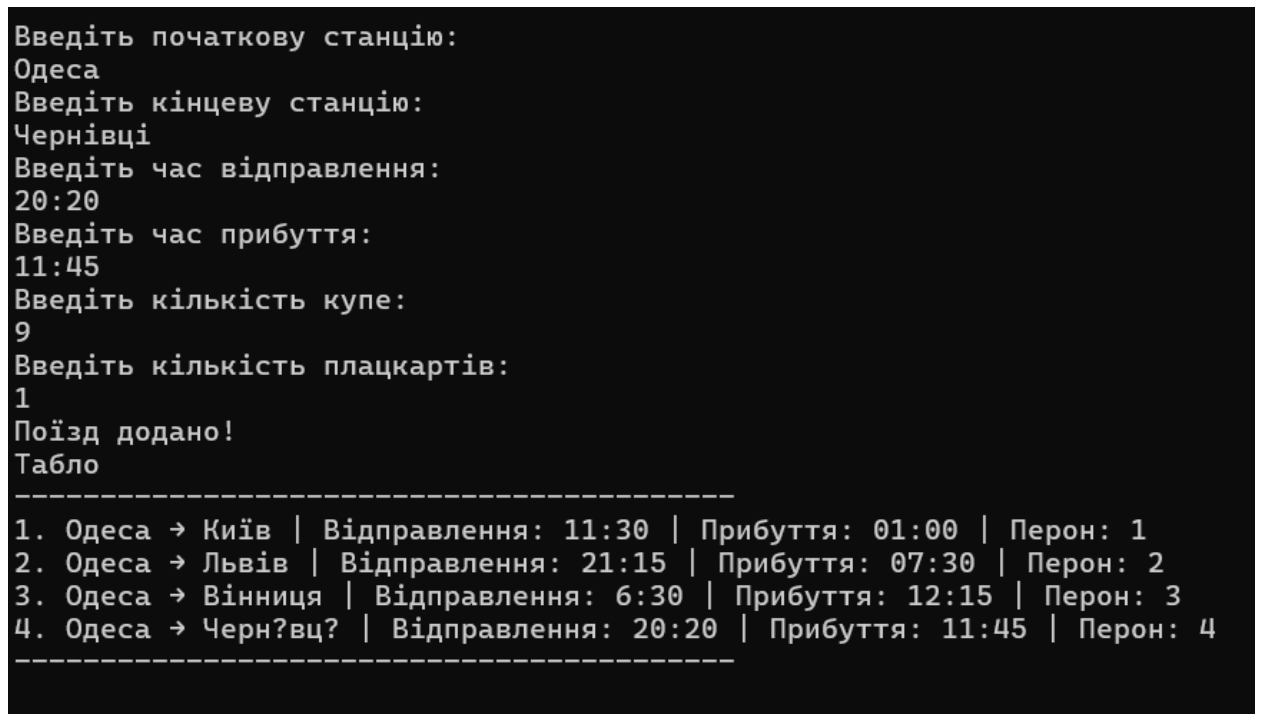


Рисунок 2.9 – додавання нового потягу та оновлене табло

У випадку вибору користувачем варіанту 2 на панелі диспетчера (рис. 2.8). Видалення потягу, користувач також побачить табло, і йому буде запропоновано ввести номер потягу, що відбув. Після вірного введення номеру потягу користувач отримає повідомлення, що потяг відбув, і побачить оновлене табло (Рис. 2.10). Програма запропонує купити квиток.

```

Табло
-----
1. Одеса → Київ | Відправлення: 11:30 | Прибуття: 01:00 | Перон: 1
2. Одеса → Львів | Відправлення: 21:15 | Прибуття: 07:30 | Перон: 2
3. Одеса → Вінниця | Відправлення: 6:30 | Прибуття: 12:15 | Перон: 3
-----

Введіть номер поїзду, що відбув
1
Поїзд відбув. Перон звільнено.
Табло
-----
1. Одеса → Львів | Відправлення: 21:15 | Прибуття: 07:30 | Перон: 2
2. Одеса → Вінниця | Відправлення: 6:30 | Прибуття: 12:15 | Перон: 3
-----

```

Рисунок 2.10 – видалення потягу та оновлене табло

ВИСНОВКИ

У ході виконання курсової роботи було створено програму мовою С# з застосуванням концепцій об'єктно-орієнтованого програмування на тему «Залізничний вокзал». В процесі розробки була побудована ієрархія класів для моделювання роботи вокзалу, поїздів, вагонів та пасажирів, а також реалізована взаємодія між ними.

При написанні програми були задіяні ключові принципи ООП: інкапсуляція даних через властивості з валідацією, наслідування на прикладі абстрактного класу «Carriage» та його підкласів «CompartmentCar» і «SittingCar», а також поліморфізм при роботі з різними типами вагонів через базовий тип. Для кожного класу були визначені відповідні поля, властивості з перевіркою вхідних даних, конструктори та методи.

Було реалізовано метод прибуття поїзда з автоматичним розподілом на вільний перон за допомогою булевого масиву, систему купівлі квитків з введенням даних пасажирів, а також панель диспетчера з можливістю додавання та видалення поїздів. У результаті виконання курсової роботи були закріплені практичні навички програмування на С#, роботи з класами, колекціями, абстрактними класами, методами, властивостями та консольним інтерфейсом. Розроблена програма демонструє можливість побудови повноцінної системи управління залізничним вокзалом засобами об'єктно-орієнтованого програмування.

СПИСОК ЛІТЕРАТУРИ

1. О.С. Антоненко, Основні принципи ООП. Лекційні матеріали. [Електронне видання] – Режим доступу: <https://docs.google.com/presentation/d/1HZ5O31Gwt5kV1TGlOY98AWB M85BtZk95/edit?slide=id.p1#slide=id.p1>
2. Є.О. Віктор, О.А. Геренко, І.М. Шпінарева, Практикум з курсу Об'єктноорієнтоване програмування мовою С#. [Електронне видання] / Одеса, 2026. – Режим доступу: <https://oop-practicum-csharp-851f60.gitlab.io/index.html>
3. Репозиторій GitHub на курсову роботу – Режим доступу: <https://github.com/chivan81/railwayStation-coursework>

Додаток А. Код класу

```
using System;
using System.ComponentModel.DataAnnotations;
using static System.Collections.Specialized.BitVector32;

namespace coursework
{
    class Program
    {
        static void Main()
        {
            Console.OutputEncoding =
System.Text.Encoding.UTF8;

            Station Odessa = new Station("Одеський вокзал",
7);

            Train train1 = new Train("Одеса", "Київ",
"11:30", "01:00", 15, 0);
            Train train2 = new Train("Одеса", "Львів",
"21:15", "07:30", 11, 0);
            Train train3 = new Train("Одеса", "Вінниця",
"6:30", "12:15", 10, 0);

            train1.AddCarriages(7, 8);
            train2.AddCarriages(5, 6);
            train3.AddCarriages(3, 7);
            Odessa.ArriveTrain(train1);
            Odessa.ArriveTrain(train2);
            Odessa.ArriveTrain(train3);

            bool isValid = false;
            bool isDispatcher = false;
            bool ticketBuy = false;

            while (!isValid)
```

Лістинг А.1 — клас Program, аркуш 1

```

        {
            Ticket ticket = null;
            Console.WriteLine("Виберіть дію:");
            Console.WriteLine("1. Одеський вокзал
(купити квиток)");
            Console.WriteLine("2. Панель диспетчера");
            string choice = Console.ReadLine();

            switch (choice)
            {
                case "1":
                    Console.Clear();
                    Console.WriteLine(Odessa);
                    int trainNum;
                    Console.WriteLine("Введіть номер
потягу:");

                    while (true)
                    {
                        if
(int.TryParse(Console.ReadLine(), out trainNum)
                        && trainNum >= 1 &&
trainNum <= Odessa.Trains.Count())
                        {
                            break;
                        }

                        Console.WriteLine("Не вірне
значення номеру потягу, спробуйте ще раз");
                    }

                    Train selectedTrain =
Odessa.Trains[trainNum - 1];

                    Console.Clear();
                    Console.WriteLine(selectedTrain);

```

```

        Console.WriteLine(selectedTrain.CarriagesCount());
        Console.WriteLine("Введіть тип
вагону: \n 1. Купе\n 2. Плацкарт");
        string choiseCar;
        while (true)
        {
            choiseCar = Console.ReadLine();
            if (choiseCar == "1" ||
choiseCar == "2")
                break;
            Console.WriteLine("Невірний
вибір! Спробуйте ще раз.");
        }
        switch (choiseCar)
        {
            case "1":
                Console.Clear();
                int comNum;
                Console.WriteLine($"Введіть
номер вагону \n Всього: {selectedTrain.compartmentCount()}");
                while (true)
                {
                    if
(int.TryParse(Console.ReadLine(), out comNum)
                        && comNum >= 1 &&
comNum <= selectedTrain.compartmentCount())
                    {
                        break;
                    }
                    Console.WriteLine("Не
вірне значення номеру вагону, спробуйте ще раз");
                }
                Carriage selectedCom =
selectedTrain.Carriages[comNum - 1];

```

```

        Console.WriteLine(selectedCom.CarInfo());
        Console.WriteLine("Купити
квиток?\n 1. Y   2. N");

        char comTicket;
        while (true)
        {
            if
(char.TryParse(Console.ReadLine(), out comTicket)
                        && (comTicket
== 'Y' || comTicket == 'N'))
            {
                break;
            }

        Console.WriteLine("Спробуйте ще раз.");
        }
        switch (comTicket)
        {
            case 'Y':
                if
(selectedCom.FreePlaces() > 0)
                {

        Console.Clear();

                                ticket = new
Ticket(selectedTrain.ToString(), "Купе", comNum);

        selectedCom.ReducePlace();

                                ticketBuy =
true;

                                break;
                }
            else

```

```

        {

selectedCom.ReducePlace();

                                break;
        }
        case 'N':
            Console.Clear();
            break;
        default:

Console.WriteLine("Невірний вибір! Спробуйте ще раз.");
                                break;
    }
    break;
    case "2":
        Console.Clear();
        int sitNum;
        Console.WriteLine($"Введіть
номер вагону \n Всього: {selectedTrain.sittingCount()}");
        while (true)
        {
            if
(int.TryParse(Console.ReadLine(), out sitNum)
                                && sitNum >= 1 &&
sitNum <= selectedTrain.sittingCount())
            {
                break;
            }
            Console.WriteLine("Не
вірне значення номеру вагону, спробуйте ще раз");
        }

        Carriage selectedSit =
selectedTrain.Carriages[selectedTrain.compartmentCount() +
sitNum - 1];

```

```

Console.WriteLine(selectedSit.CarInfo());

                                                                    Console.WriteLine("Купити
квиток?\n 1. Y   2. N");

                                                                    char sitTicket;
                                                                    while (true)
                                                                    {
                                                                    if
(char.TryParse(Console.ReadLine(), out sitTicket)
                                                                    && (sitTicket
== 'Y' || sitTicket == 'N'))
                                                                    {
                                                                    break;
                                                                    }

Console.WriteLine("Спробуйте ще раз.");
                                                                    }
                                                                    switch (sitTicket)
                                                                    {
                                                                    case 'Y':
                                                                    if
(selectedSit.FreePlaces() > 0)
                                                                    {

Console.Clear();

                                                                    ticket = new
Ticket(selectedTrain.ToString(), "Пляцкарт", sitNum);

selectedSit.ReducePlace();

                                                                    ticketBuy =
true;

                                                                    break;
                                                                    }
                                                                    else

```

```

{
selectedSit.ReducePlace();

break;
}
case 'N':
Console.Clear();
break;
default:

Console.WriteLine("Невірний вибір! Спробуйте ще раз.");
break;
}
break;
}
break;
case "2":
Console.Clear();
Console.WriteLine(Odessa);
isDispatcher = true;
Console.WriteLine("Панель
диспетчера:");

Console.WriteLine("1. Додати
поїзд");

Console.WriteLine("2. Видалити
поїзд");

string dispChoice;
while (true)
{
dispChoice =
Console.ReadLine();

if (dispChoice == "1" ||
dispChoice == "2")
{

```

```

        break;
    }
    Console.WriteLine("Спробуйте ще
раз.");

    }
    switch (dispChoice)
    {
        case "1":
            {
                try
                {
                    Console.Clear();

Console.WriteLine("Введіть початкову станцію:");
                    string startSt =
Console.ReadLine();

Console.WriteLine("Введіть кінцеву станцію:");
                    string endSt =
Console.ReadLine();

Console.WriteLine("Введіть час відправлення:");
                    string depTime =
Console.ReadLine();

Console.WriteLine("Введіть час прибуття:");
                    string arrTime =
Console.ReadLine();

                    int compCount;
                    while (true)

```



```

        {

Console.WriteLine("Введіть кількість купе:");

                                if
(int.TryParse(Console.ReadLine(), out compCount) && compCount >=
1)

                                break;

Console.WriteLine("Невірне значення!");

        }

        int sitCount;
        while (true)
        {

Console.WriteLine("Введіть кількість плацкартів:");

                                if
(int.TryParse(Console.ReadLine(), out sitCount) && sitCount >=
1)

                                break;

Console.WriteLine("Невірне значення!");

        }

        Train newTrain =
new Train(startSt, endSt, depTime, arrTime, 0, 0);

newTrain.AddCarriages(compCount, sitCount);

Odessa.ArriveTrain(newTrain);

Console.WriteLine("Поїзд додано!");

Console.WriteLine(Odessa);

                                isValid = true;
                                break;

```

```

    }
    catch
    (ArgumentException ex)
    {
        Console.WriteLine($"Помилка: {ex.Message}");
        break;
    }
    catch (FormatException)
    {
        Console.WriteLine("Помилка: введіть коректні дані!");
        break;
    }
}
case "2":
{
    Console.Clear();

    Console.WriteLine(Odessa);

    int delNum;
    while (true)
    {
        Console.WriteLine("Введіть номер поїзду, що відбув");
        if
        (int.TryParse(Console.ReadLine(), out delNum)
        && (delNum >= 1
        && delNum <= Odessa.Trains.Count))
        break;

        Console.WriteLine("Спробуйте ще раз.");
    }
}

```

```

Odessa.DepartureTrain(delNum);

Console.WriteLine("Поїзд відбув. Перон звільнено.");

Console.WriteLine(Odessa);

                                isValid = true;
                                break;
                                }
                                }

                                break;
                                default:
                                    Console.WriteLine("Невірний вибір!
Спробуйте ще раз.");
                                    break;
                                }
                                Passanger passanger = null;
                                bool validPass = false;
                                if (ticket != null)
                                {
                                    while (ticketBuy == true)
                                    {
                                        try
                                        {
                                            Console.WriteLine("Введіть
ПІБ:");

                                            string name =
Console.ReadLine();

                                            Console.WriteLine("Введіть
стать (М/Ф):");

                                            char gender =
char.Parse(Console.ReadLine());

```

```

        Console.WriteLine("Введіть свій
вік:");

        int age =
int.Parse(Console.ReadLine());

        Console.WriteLine("Введіть
номер паспорту:");

        string id = Console.ReadLine();

        passanger = new Passanger(name,
gender, age, id);

        ticketBuy = false;
        break;
    }
    catch (ArgumentException ex)
    {
        Console.WriteLine($"Помилка:
{ex.Message}");
    }
    catch (FormatException)
    {
        Console.WriteLine("Помилка:
введіть коректне число!");
    }
}

Console.Clear();
Console.WriteLine(passanger);
Console.WriteLine(ticket);
}

else if (!isDispatcher)
{ Console.WriteLine("Покупку скасовано."); }

Console.WriteLine("\nБажаєте купити ще
квиток? (Y/N)");

```

```

        char again;
        while (true)
        {
            if (char.TryParse(Console.ReadLine(),
out again)
                && (again == 'Y' || again == 'N'))
                break;
            Console.WriteLine("Спробуйте ще раз.");
        }
        if (again == 'Y')
            isValid = false;
        else
            isValid = true;
    }
}
}
}

```

Лістинг A.1 — клас Program, аркуш 13

```

using System;
using System.ComponentModel.DataAnnotations;

namespace coursework
{
    class Station
    {
        private string name;
        private int platformAmount;
        private bool[] platforms;
        private List<Train> trains;
    }
}

```

Лістинг A.2 — Клас Station, аркуш 1

```

public Station() : this(" ", 1) { }

public List<Train> Trains
{
    get { return trains; }
}

public string Name
{
    get { return name; }
    set
    {
        if (string.IsNullOrEmpty(value))
            throw new ArgumentException("Назва
вокзалу не може бути порожньою!");
        name = value;
    }
}

public int PlatformAmount
{
    get { return platformAmount; }
    set
    {
        if (value < 1)
            throw new ArgumentException("Перонів не
може бути менше одного");
        platformAmount = value;
    }
}

```

```
public Station(string name, int platformAmount)
{
    Name = name;
    PlatformAmount = platformAmount;
    platforms = new bool[platformAmount];
    trains = new List<Train>();
}

public void ArriveTrain(Train train)
{
    for (int i = 0; i < platforms.Length; i++)
    {
        if (platforms[i] == false)
        {
            platforms[i] = true;
            train.Platform = i + 1;
            trains.Add(train);
            return;
        }
    }
    Console.WriteLine("Всі перони зайняті!");
}

public void DepartureTrain(int delNum)
{
    Train train = trains[delNum - 1];
    platforms[train.Platform - 1] = false;
    trains.RemoveAt(delNum - 1);
}
```

```

private string TrainsInfo()
{
    string result = "";
    for (int i = 0; i < trains.Count; i++)
        result += $"{i + 1}. {trains[i]}\n";
    return result;
}

public override string ToString()
{
    return $"Табло\n" +

    "-----\n" +

        TrainsInfo() +

    "-----\n";
}
}

```

Лістинг А.2 — Клас Station, аркуш 4

```

using System;
using System.ComponentModel.DataAnnotations;
namespace coursework
{
    class Train
    {
        private string startStation;
        private string finalStation;
        private string departureTime;
    }
}

```

Лістинг А.3 — Клас Train, аркуш 1


```

private string arrivalTime;
private int carAmount;
private int platform;
private List<Carriage> carriages;

public Train() : this(" ", " ", null, null, 1, 0) {
}

public string StartStation
{
    get { return startStation; }
    set
    {
        if (string.IsNullOrEmpty(value))
            throw new ArgumentException("Назва
вокзалу не може бути порожньою!");
        startStation = value;
    }
}

public string FinalStation
{
    get { return finalStation; }
    set
    {
        if (string.IsNullOrEmpty(value))
            throw new ArgumentException("Назва
вокзалу не може бути порожньою!");
        finalStation = value;
    }
}

```

Лістинг А.3 — Клас Train, аркуш 2

```

public string DepartureTime
{
    get { return departureTime; }
    set
    {
        if (string.IsNullOrEmpty(value))
            throw new ArgumentException("Час
відправлення не може бути порожнім!");
        departureTime = value;
    }
}

public string ArrivalTime
{
    get { return arrivalTime; }
    set
    {
        if (string.IsNullOrEmpty(value))
            throw new ArgumentException("Час
прибуття не може бути порожнім!");
        arrivalTime = value;
    }
}

public int CarAmount
{
    get { return carAmount; }
    set
    {
        if (value < 0)

```

Лістинг А.3 — Клас Train, аркуш 3

```

        throw new ArgumentException("Вагонів не
може бути менше 0");
        carAmount = value;
    }
}
public int Platform
{
    get { return platform; }
    set
    {
        if (value < 0)
            throw new ArgumentException("Невірне
значення перону!");
        platform = value;
    }
}

public List<Carriage> Carriages => carriages;
public void AddCarriages(int compartmentCount, int
sittingCount)
{
    for (int i = 0; i < compartmentCount; i++)
        carriages.Add(new CompartmentCar());
    for (int i = 0; i < sittingCount; i++)
        carriages.Add(new SittingCar());
}

public Train(string startStation, string
finalStation, string departureTime, string arrivalTime, int
carAmount, int platform)

```

```

    {
        StartStation = startStation;
        FinalStation = finalStation;
        DepartureTime = departureTime;
        ArrivalTime = arrivalTime;
        CarAmount = carAmount;
        Platform = platform;
        carriages = new List<Carriage>();
    }

    public int compartmentCount()
    {
        return Carriages.Count(c => c is
CompartmentCar);
    }

    public int sittingCount()
    {
        return Carriages.Count(s => s is SittingCar);
    }

    public string CarriagesCount()
    {
        return
"-----\n" +
        $"Кількість вагонів різних типів: \n" +
        $"Купе: | Плацкарт:\n" +

```

```

        $"{compartmentCount()} |
{sittingCount()}\n" +

"-----\n";

    }

    public override string ToString()
    {
        return $"{StartStation} → {FinalStation} |
Відправлення: {DepartureTime} | Прибуття: {ArrivalTime} | Перон:
{Platform}";
    }
}

```

Лістинг А.3 — Клас Train, аркуш 6

```

using System;
using System.ComponentModel.DataAnnotations;

namespace coursework
{
    public abstract class Carriage
    {
        protected int totalPlaces;
        protected int freePlaces;
        protected string price;

        public int TotalPlaces() { return totalPlaces; }
        public int FreePlaces() { return freePlaces; }
        public void ReducePlace()
        {

```

Лістинг А.4 — клас Carriage, аркуш 1

```

        if (freePlaces > 0)
            freePlaces--;
        else
            Console.WriteLine("Вільних місць немає!");
    }

    public abstract string CarInfo();
}
}

```

Лістинг А.4 — клас Carriage, аркуш 2

```

using System;
using System.ComponentModel.DataAnnotations;

namespace coursework
{
    public class CompartmentCar : Carriage
    {
        public CompartmentCar()
        {
            totalPlaces = 40;
            freePlaces = 40;

            price = "800₾";
        }

        public override string CarInfo()
        {
            return $"Всього місць: {totalPlaces}\n" +

```

Лістинг А.5 — клас CompartmentCar, аркуш 1

```

        $"Вільних місць: {freePlaces}\n" +
        $"Ціна квитка: {price}";
    }
}
}

```

Лістинг A.5 — клас CompartmentCar, аркуш 2

```

using System;
using System.ComponentModel.DataAnnotations;

namespace coursework
{
    public class SittingCar : Carriage
    {
        public SittingCar()
        {
            totalPlaces = 60;
            freePlaces = 60;
            price = "400₴";
        }

        public override string CarInfo()
        {
            return $"Всього місць: {totalPlaces}\n" +
                $"Вільних місць: {freePlaces}\n" +
                $"Ціна квитка: {price}";
        }
    }
}

```

Лістинг A.6 — клас SittingCar

```
using System;

using System.ComponentModel.DataAnnotations;

using static
System.Runtime.InteropServices.JavaScript.JSType;

namespace coursework
{
    public class Passanger
    {
        private string fullName;
        private char gender;
        private int age;
        private string id;
        public string FullName
        {
            get { return fullName; }
            set
            {
                int spaces = 0;

                for (int i = 0; i < value.Length; i++)
                {
                    if (value[i] == ' ')
                        spaces += 1;
                }

                if (spaces < 2)
                    throw new
ArgumentException("Неправильно введено повне ім'я");
            }
        }
    }
}
```

Лістинг А.7 — клас Passanger, аркуш 1


```

        fullName = value;
    }
}

public char Gender
{
    get { return gender; }
    set
    {
        char lower = char.ToLower(value);
        if (lower != 'm' && lower != 'f')
            throw new
ArgumentException("Неправильно введено гендер");
        gender = lower;
    }
}

public int Age
{
    get { return age; }
    set
    {
        if (value <= 14)
            throw new ArgumentException("Ви не
можете купити білет якщо вам менше 14ти");
        age = value;
    }
}

public string Id
{
    get { return id; }
    set

```

Лістинг А.7 — клас Passanger, аркуш 2

```

        {
            if (value.ToString().Length != 9)
                throw new ArgumentException("Номер
паспорту введено неправильно.");
            id = value;
        }
    }

    public Passanger(string fullName, char gender, int
age, string id)
    {
        FullName = fullName;
        Gender = gender;
        Age = age;
        Id = id;
    }

    public override string ToString()
    {
        string stat = Gender == 'm' ? "чоловік" :
"жінка";

        return $"Ваш білет:\n" +

"-----\n" +
        $"ПІБ: {FullName}\n" +
        $"Стать: {stat}\n" +
        $"Вік: {Age} \n" +
        $"Номер паспорту: {Id} \n" +

```

Лістинг А.7 — клас Passanger, аркуш 3

```
"-----\n";
```

```
    }
}
}
```

Лістинг А.7 — клас Passanger, аркуш 4

```
using System;
using System.ComponentModel.DataAnnotations;

namespace coursework
{
    public class Ticket
    {
        private string selectedTrain;
        private string selectedCar;
        private int carNum;

        public string SelectedTrain
        {
            get { return selectedTrain; }
            set { selectedTrain = value; }
        }
        public string SelectedCar
        {
            get { return selectedCar; }
            set { selectedCar = value; }
        }
    }
}
```

Лістинг А.8 — клас Ticket, аркуш 1

```

public int CarNum
{
    get { return carNum; }
    set { carNum = value; }
}

public Ticket(string selectedTrain, string
selectedCar, int carNum)
{
    SelectedTrain = selectedTrain;
    SelectedCar = selectedCar;
    CarNum = carNum;
}

public override string ToString()
{
    return
        $"Пейс: \n" +
        SelectedTrain +
        $"\nТип вагону | Номер вагону \n" +
        $"{SelectedCar} | {CarNum} ";
}
}
}

```

Лістинг А.8 — клас Ticket, аркуш 2

Додаток Б. Діаграма класів

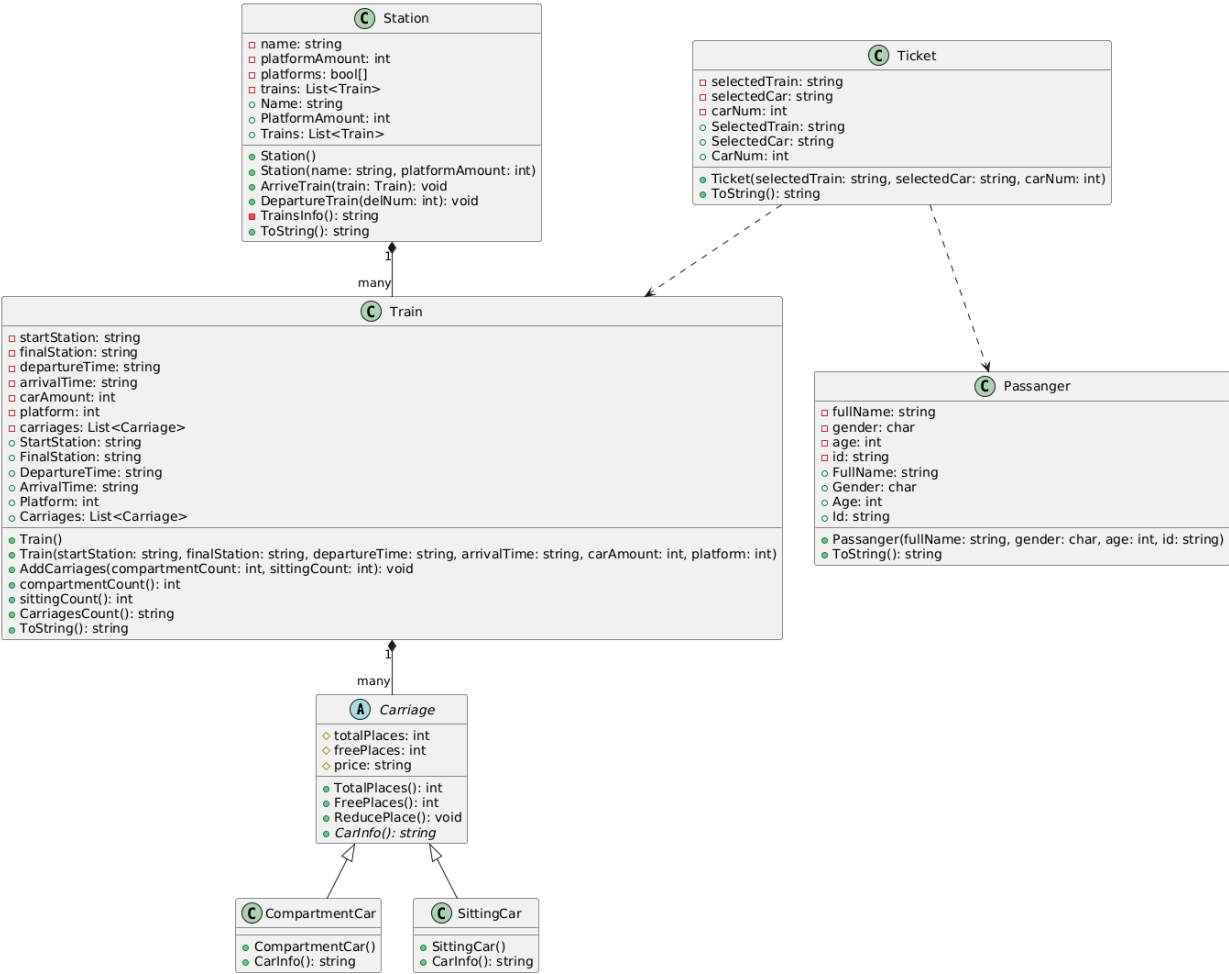


Рисунок Б.2 — Діаграма класів